Plano de Ação: Divisão de Responsabilidades da Refatoração MVVM

Estruturação, Eliminação de Redundâncias e Estabilidade · Portal FCV

ENGENHARIA DE SOFTWARE

Escopo: todo o `src/` (models, viewmodels, pages, components, styles, services), configs e README.

Diagnóstico Base: `tsc -b` ✅ sem erros · `npm test` ✅ 9/9 · `vite build` ✅ (403 kB JS / 51 kB CSS).

> **Diretriz:** O projeto compila e os testes passam. As responsabilidades abaixo atacam diretamente os gargalos identificados na análise: a sobrecarga de responsabilidades nas páginas, ausência de ViewModels por tela, seletores com custo quadrático, redundâncias de CSS e bugs de navegação.

1. Distribuição de Responsabilidades

Kaio — Camada Model — `src/models/`

Foco de Atuação: Reestruturar a camada Model, separando contratos de dados, regras de negócio puras, formatadores de apresentação e otimizando algoritmos de consulta.

- **Aprimoramento de Contratos de Tipos:** Refinar `types.ts` tipando os enums de status (colaborador, conveniado, campanha, utilização), regras de ciclo e interfaces de relatórios (`ReportResult`).
- **Isolamento de Formatadores:** Criar `formatters.ts` centralizando formatações (moeda, data, CNPJ, status e iniciais), com tratamento defensivo para datas no formato SQL (`replace(" ", "T")`).
- **Extração de Regras de Domínio:** Criar `rules.ts` isolando regras de permissão de administrador, validação de estorno (`canEstornar`, `validateEstorno`) e checagem de períodos.
- **Otimização de Seletores e Joins:** Em `selectors.ts`, focar em relacionamento de entidades e manter re-exportações para garantir 100% de retrocompatibilidade com as páginas existentes.
- **Agregação Eficiente em Relatórios:** Em `reportSelectors.ts`, substituir buscas lineares aninhadas por indexação via `Map`/`Set`, reduzindo a complexidade de $O(N^2)$ para $O(N)$.
- **Ponto de Entrada da Camada:** Criar `index.ts` como barrel export oficial da camada Model.

Arquivos de atuação: `src/models/types.ts`, `src/models/formatters.ts`, `src/models/rules.ts`, `src/models/selectors.ts`, `src/models/reportSelectors.ts`, `src/models/index.ts`

Pedro — Camada ViewModel & Serviços — `src/viewmodels/ & src/services/`

Foco de Atuação: Desacoplar estado global, isolar ViewModels dedicados por tela e garantir robustez na comunicação de rede e ciclo de vida.

- **Desacoplamento do Estado Global:** Separar a lógica de toasts e notificações de `PortalContext.tsx` criando um `ToastContext.tsx`, eliminando re-renders desnecessários em cascata a cada aviso.
- **Memoização de Contextos:** Envolver valores de contexto em `useMemo` e handlers em `useCallback` para estabilidade de render.
- **Criação de ViewModels por Página:** Extrair a lógica de negócio e estados que residem nas páginas grandes (500 a 1100 linhas) para custom hooks específicos (ex.: `useColaboradoresVM`, `useUtilizacoesVM`).
- **Correção de Remontagem de Rotas (Bug B1):** Eliminar a dependência de `location.search` na key do `<Outlet />` em `Layout.tsx` para não resetar filtros ao fechar modais ou atualizar parâmetros de URL.
- **Tratamento de Payload Parcial na API (Bug B2):** Tratar o comportamento de `PUT` em `api.ts` (inativação/descontinuação) para garantir que campos não enviados não sejam corrompidos no backend.

Arquivos de atuação: `src/viewmodels/PortalContext.tsx`, `src/viewmodels/ToastContext.tsx`, `src/viewmodels/hooks/*`, `src/components/Layout.tsx`, `src/services/api.ts`

Camilly — Camada View & Experiência de Uso — `src/pages/ & src/components/`

Foco de Atuação: Refatorar telas para torná-las puramente declarativas, padronizar feedback de formulários e garantir usabilidade e acessibilidade.

- **Consistência de Feedback Visual:** Unificar tratamento de erros: substituir toasts aleatórios de validação por avisos inline consistentes nos formulários de cadastro.
- **Correção da Paginação de Campanhas (Bug B4):** Ajustar `Campanhas.tsx` para que o fatiamento de página ocorra individualmente em cada tabela (Campanhas vs. Eventos), evitando listas vazias artificiais.
- **Padronização de Modais de Confirmação:** Criar um componente padrão `ConfirmModal` para inativação e estorno, eliminando blocos redundantes de modal nas telas.
- **Acessibilidade & Microinterações:** Revisar navegação por teclado (foco e tecla Escape), atributos ARIA em `Modal`, `PortalSearch` e `NotificationCenter`.
- **Emagrecimento das Telas:** Conectar as páginas aos ViewModels preparados pelo Pedro, tornando as views declarativas e focadas apenas na renderização da interface.

Arquivos de atuação: `src/pages/*.tsx`, `src/components/ui.tsx`, `src/components/PortalSearch.tsx`, `src/components/NotificationCenter.tsx`

Diogo — Estrutura CSS, Limpeza & Helpers — `src/styles/ & src/components/`

Foco de Atuação: Sanar redundâncias de estilização, criar utilitários reutilizáveis de suporte e eliminar código morto do projeto.

- **Unificação de CSS (41 seletores duplicados):** Consolidar regras repetidas entre `portal.css`, `reference.css` e `refinements.css`, removendo declarações conflitantes e `!important`.
- **Hook de Paginação Reutilizável:** Criar o hook `usePagination(total, pageSize)` para substituir os blocos repetidos de cálculo de página em 7 telas diferentes.
- **Remoção de Código Morto e Tipos Órfãos:** Limpar aliases não utilizados (como `PerfilAcesso`) e variáveis sem uso.
- **Ajustes de Qualidade de Código:** Corrigir comparações não estritas (ex.: `!=` para `!==` em `api.ts`) e padronizar ícones no `PageHeader`.
- **Estados Vazios Padronizados:** Assegurar que todas as tabelas e listas utilizem `EmptyState` de forma uniforme.

Arquivos de atuação: `src/styles/portal.css`, `src/styles/reference.css`, `src/styles/refinements.css`, `src/components/ui.tsx`, `src/hooks/usePagination.ts`

2. Matriz de Alinhamento e Dependências

Etapa	Responsável	Entrada / Dependência	Impacto Principal
1. Model & Contratos	Kaio	Independente	Garante a base de tipos, regras de domínio e otimização dos cálculos.
2. Limpeza CSS & Helpers	Diogo	Independente	Reduz peso do bundle, limpa CSS e fornece hook de paginação.
3. ViewModels & Contextos	Pedro	Model (Kaio)	Isola a lógica de negócio das telas e corrige bugs de rota/API.
4. UX & Refatoração de Views	Camilly	ViewModels (Pedro) & Helpers (Diogo)	Fecha as interfaces, melhora acessibilidade e finaliza o MVVM.

Critérios de Aceitação Gerais (Definition of Done):

✓ Todo código novo ou refatorado deve compilar sem erros no TypeScript (`tsc -b`).

✓ Nenhuma lógica de cálculo ou regra de negócio deve residir diretamente no JSX das Views.

✓ Commits devem seguir mensagens semânticas e concisas (ex.: `refactor(viewmodel): ...`, `style(css): ...`).